Assessment Requirements

Section 1: Control Structures and Importing Data (12 marks)

Description: **Implement control structures (if statements, loops etc.) to analyse data.**

Task: Select a publicly available dataset (e.g., from Kaggle, UCI Machine Learning Repository, or government databases) related to a real-world problem (e.g., climate change, public health, or economic indicators. An example of datasets is given at the end). Write code within a Jupyter Notebook that uses control structures to import the data from a sharable link (e.g., from the actual source of the data, or stored in OneDrive/Dropbox), process the data, deriving insights such as trends, averages, or anomalies based on specific conditions.

Deliverables:

Jupyter Notebook (.ipynb): Submit a well-documented Jupyter Notebook with clear code, explanations, and analysis using control structures. Include markdown cells as part of the written report to describe the dataset, methods, and key findings. Ensure all results are displayed within the notebook (Find the given .ipnyb Template from NOW).

Written Report: The markdown cells within the Jupyter Notebook will summarise the dataset and real-world problem. Explain how control structures were used, provide key findings from the analysis, and include a reflection on results and limitations.

Code Quality: Ensure the code is clean, readable, and follows best practices. Use meaningful variable names, appropriate indentation, and clear logic, adhering to Python coding standards.

Section 2: Functions and Modules (16 marks)

Description: **Design and implement functions and use modules effectively.**

Task: In the same Jupyter Notebook, create multiple functions to handle different aspects of data analysis for the selected dataset. Additionally, import and utilize at least 3 or 4 external modules (e.g., `requests` for API calls, `NumPy` for numerical operations) to enhance the program's capabilities. Ensure the functions are reusable, well-documented, and modular.

Deliverables:

Jupyter Notebook (.ipynb): *Use the same notebook as section 1*, expanded to include multiple well-structured functions for different aspects of data analysis. Use external modules effectively, ensuring proper imports and clear integration into the analysis process. The functions should be documented within markdown cells to explain their purpose and usage.

Written Report: The markdown cells in the notebook will also serve as the report, detailing the functions, the rationale for modularizing the code, and the use of external modules. Provide explanations for the functions created, the role of each module, and how they contribute to the overall analysis.

Code Quality: Maintain high code quality, with attention to modularity, readability, and reusability. Ensure appropriate commenting, meaningful variable names, and adherence to Python coding standards throughout.

Section 3: Data Handling with Pandas (16 marks)

Description: **Use Pandas for data manipulation and analysis.**

Task: Analyse the dataset using Pandas within the Jupyter Notebook. Perform tasks such as data cleaning (handling missing values and duplicates), filtering, and aggregating data to derive meaningful insights. Document all steps clearly using markdown and comments to explain the logic and methodology.

Deliverables:

Jupyter Notebook (.ipynb): Continue using the same notebook to implement data manipulation and analysis using Pandas. Include tasks like data cleaning, filtering, and aggregating to extract insights. Document the logic and workflow in markdown cells, ensuring clarity in the steps taken for each analysis.

Written Report: The markdown documentation within the notebook serves as the report. Provide detailed explanations of the data handling process, including how missing values, duplicates, and other inconsistencies were managed. Discuss the results of filtering and aggregation, linking them to the overall analysis objectives.

Code Quality: Ensure code adheres to good coding practices with organised and readable Pandas operations. Comment clearly, and use descriptive variable names to enhance understanding of the data processing steps.

Section 4: Data Visualization (16 marks)

Description: **Create visual representations of data and develop a simple graphical user interface (GUI).**

Data Visualization: Using Matplotlib or Seaborn, generate at least five to six different types of plots to visualise trends or relationships within the dataset. Incorporate customisations such as subplots or interactive plots (using Plotly or Seaborn). Ensure that the visualisations include appropriate titles, labels, legends, and annotations with an inclusive choice of colors.

Deliverables:

Jupyter Notebook (.ipynb): In the Jupyter Notebook (.ipynb), you will use Matplotlib or Seaborn to create five to six different types of plots to visualize trends or relationships within the dataset. It is essential to document your code clearly in markdown cells, explaining each visualization and the logic behind the choice of plots, as well as any customizations made, such as colors, labels, and legends. Each plot should include appropriate titles, axis labels, legends, and annotations where necessary, ensuring an inclusive choice of colors for accessibility. Please choose your visualisations carefully. *Only 'first six (06)' visualisations will be considered for marking*.

Written Report: The markdown documentation within the notebook will serve as the report, providing detailed explanations of the visualization process. You should discuss how each plot contributes to understanding the dataset and justify the choices of colors and customizations that enhance clarity and accessibility. Additionally, explain any insights gained from the visualizations and how they relate to the overall objectives of the analysis.

Code Quality: For code quality, ensure that your code adheres to good coding practices by focusing on organized and readable plot creation. Include clear comments throughout the code to explain the function of each part, and use descriptive variable names to enhance understanding of the visualization steps and choices.

Section 5: GUI Development (16 marks)

Description: **Create visual representations of data and develop a simple graphical user interface (GUI).**

GUI Development (16 marks): Create a simple Python GUI using libraries like Tkinter or PyQt to allow users to interact with the dataset. For example, the GUI might allow users to select specific data subsets or perform basic analysis tasks (e.g., displaying summary statistics, generating specific visualizations) by interacting with buttons or input fields. Focus on usability and the ability to perform core functions within the GUI.

Deliverables:

Jupyter Notebook (.ipynb): The Jupyter Notebook (.ipynb) will include the code for the GUI development, demonstrating how users can engage with the dataset. You should provide thorough documentation in markdown cells that explain your design choices, the functionality of the GUI, and any usability considerations that were taken into account during development.

Written Report: The markdown documentation within the notebook will serve as the report, detailing how the GUI facilitates user interaction with the dataset. You should explain the core functions available in the GUI and highlight the usability features that enhance the overall user experience.

Code Quality: To ensure code quality, your implementation should adhere to good coding practices. This includes an organized structure, clear comments throughout the code, and the use of descriptive variable names. Following guidelines for creating intuitive user interfaces will further enhance the usability and effectiveness of your GUI.

Section 6: Version Control, Critical Appraisal, Documentation, and Report (24 marks)

Description: Demonstrate the use of version control practically, prepare a comprehensive report, and document the code clearly.

Version Control: Set up a GitHub repository for the coursework and commit work incrementally throughout the project. *Your GitHub activity from Week-03 will be considered and monitored.* Ensure:

- Regular commits with clear messages describing changes.

- A well-structured repository (e.g., directories for data, notebooks, and documentation).

Overall Reporting: Within the same Jupyter Notebook (that you have created earlier), include:

- Introduction: Brief overview of the dataset and real-world problem at the beginning.

- Implementation/Reporting: Description of data processing and analysis techniques (Sections 1 to 5 individually as mentioned in the deliverables for individual sections).

- Findings: Key insights and visualisations (Sections 1 to 5 individually as mentioned in the deliverables for individual sections).

- Conclusion: Summary of results and implications at the end.

Code Documentation: Ensure that each section of the code is thoroughly commented to explain the purpose and logic. Use Python's docstring format for documenting functions.

Coding Standards and Critical Appraisal: Follow consistent coding practices and code design. Possible methods of critical appraisals are-

- *Code Reviews*- Peer review can help identify issues and improve code quality.
- *Static Analysis Tools*- Automated tools can detect potential problems like syntax errors, code smells, and security vulnerabilities.
- *Unit Testing*- Writing tests for individual code units can help verify correctness and identify regressions.
- *Profiling*- Analysing the performance of the code can help identify bottlenecks and areas for optimisation.